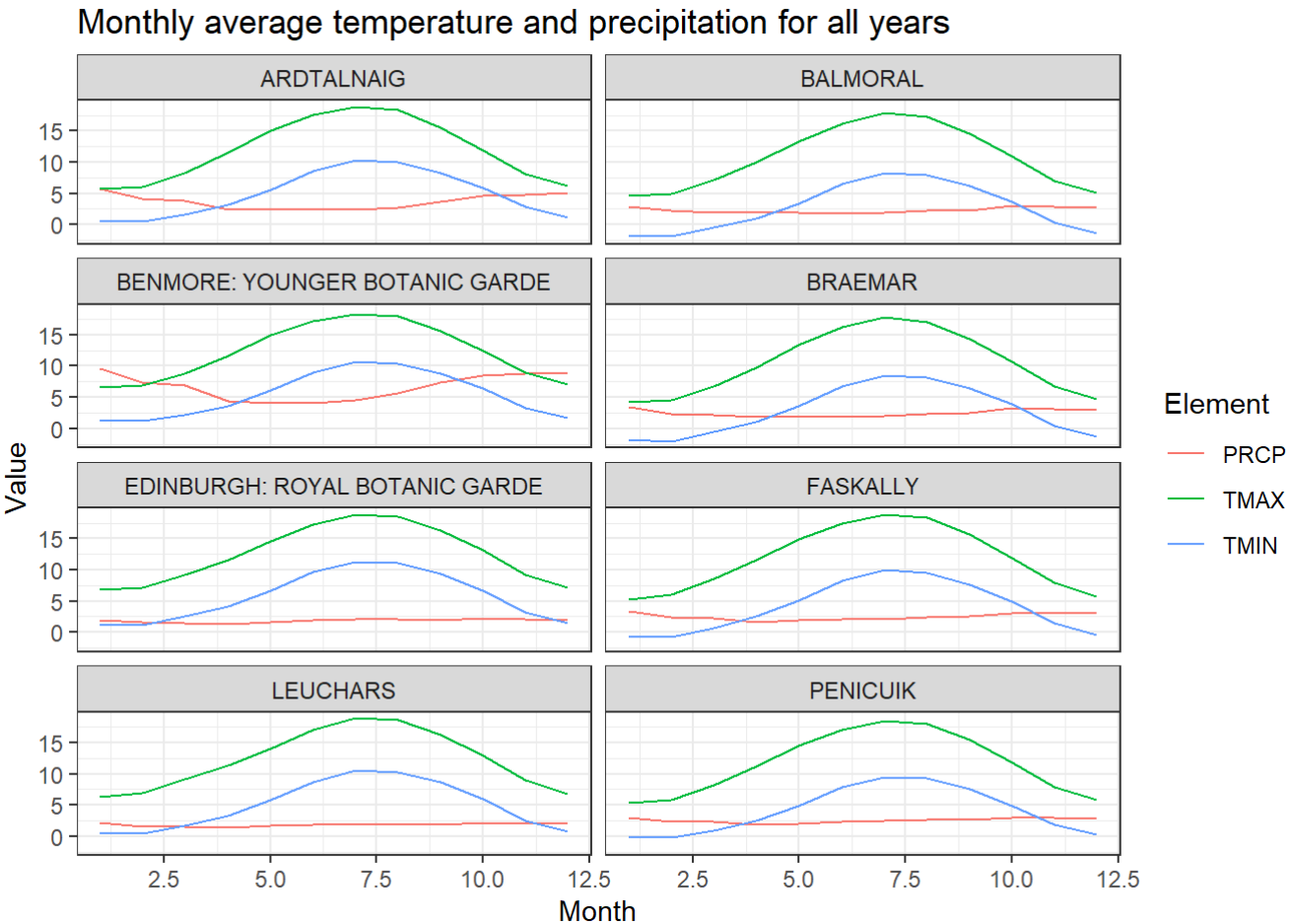
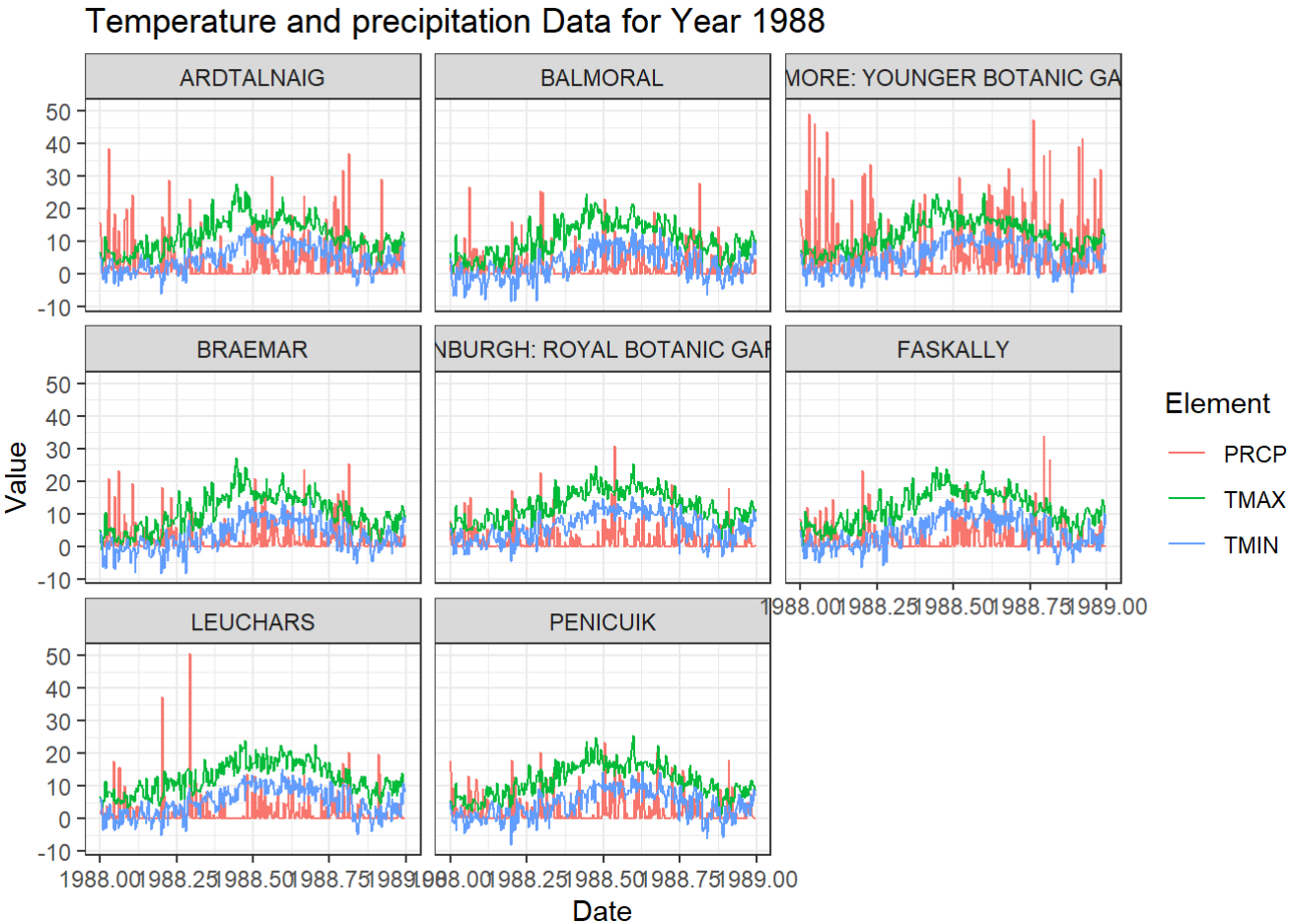


StatComp Project 2: Scottish weather

Chenrui Xu

Seasonal variability



The first graph, titled “Temperature and precipitation Data for Year 1988”, shows temperature (TMIN and TMAX) and precipitation (PRCP) data for the specific year of 1988. The graph is organized into facets based on the weather station names and arranged in a 3-column layout. Although some stations show variability and

seasonal patterns, it is not immediately evident if there is a consistent pattern across all locations.

The second graph, titled “Monthly average temperature and precipitation for all years”, shows the monthly average values for temperature (TMIN and TMAX) and precipitation (PRCP) for all years in the dataset. This graph is organized into facets based on the weather station names and arranged in a 2-column layout with a black-and-white theme.

When examining both graphs to answer the question, “Is there a seasonal effect for temperature (TMIN and TMAX) and precipitation (PRCP)? e.g., is it colder in the winter than in the summer?”, we can make the following observations:

For temperature, a clear seasonal pattern is evident in the second graph, with TMIN and TMAX lines generally following a sinusoidal pattern. In most cases, the temperature is lower in winter months and higher in summer months, confirming that it is colder in the winter than in the summer.

For precipitation, while the patterns may not be as evident in the first graph, the second graph shows some seasonal effects in certain weather stations. The precipitation levels may vary depending on the specific location, but we can see fluctuations throughout the year, with some stations showing increased precipitation during specific months or seasons.

In conclusion, the two graphs reveal a seasonal effect on both temperature and precipitation, characterized by colder temperatures in winter and warmer temperatures in summer, as well as varying precipitation levels depending on location and time of year. Furthermore, some stations, such as Ardtalnaig and Benmore Botanic Gardens, exhibit more overall variability and a discernible seasonal pattern. However, it remains unclear whether a consistent seasonal pattern exists for all locations.

```

# Load data from StatCompLab package
data("ghcnd_values", package = "StatCompLab")
data("ghcnd_stations", package = "StatCompLab")
ghcnd <- ghcnd_values %>%
  mutate(Summer = Month %in% c(4, 5, 6, 7, 8, 9)) %>%
  left_join(ghcnd_stations, by = "ID")
saveRDS(ghcnd, "data/ghcnd.rds")

# Preprocess ghcnd data
ghcnd_mon <- ghcnd_values %>%
  filter(Element == "PRCP") %>%
  group_by(ID, Year, Month) %>%
  summarise(
    DecYear = mean(DecYear),
    Value = mean(Value)^0.5,
    .groups = "drop"
  ) %>%
  left_join(ghcnd_stations, by = "ID") %>%
  incorporate_harmonics(harmonic_order = 4)
saveRDS(ghcnd_mon, "data/ghcnd_mon.rds")

ghcnd <- readRDS("data/ghcnd.rds")

# Perform seasonal permutation test
part1_timing <- system.time({
  N_T <- 1000
  Test0 <- seasonal_permutation_test_statistics(ghcnd, random = FALSE)
  Test <- Test0 %>%
    mutate(p_value = 0,
           N_T = N_T) %>%
    select(-Test)
  for (loop in seq_len(N_T)) {
    T_rand <- seasonal_permutation_test_statistics(ghcnd, random = TRUE)
    # Accumulate information about the average test statistics,
    # and p-value estimates.
    Test$p_value <- Test$p_value + (Test0$Test <= T_rand$Test) / N_T
  }
})

# Save and load data
saveRDS(list(N_T = N_T,
            timing = part1_timing), "data/part1_metadata.rds")
saveRDS(Test, "data/Test.rds")

Test <- readRDS("data/Test.rds")
# Add confidence intervals:
Test <- Test %>%
  rowwise() %>%
  mutate(p_value_confidence_interval(p_value, N_T)) %>%
  ungroup()

saveRDS(Test, "data/Test.rds")

# Save Test data
ghcnd_mon <- readRDS("data/ghcnd_mon.rds")

```

ID	Name	Element	p_value	N_T	lwr	upr
UKE00105874	BRAEMAR	PRCP	0.000	1000	0.0000000	0.0036821
UKE00105875	BALMORAL	PRCP	0.000	1000	0.0000000	0.0036821
UKE00105884	ARDTALNAIG	PRCP	0.000	1000	0.0000000	0.0036821
UKE00105885	FASKALLY	PRCP	0.000	1000	0.0000000	0.0036821
UKE00105886	LEUCHARS	PRCP	0.024	1000	0.0145141	0.0334859
UKE00105887	PENICUIK	PRCP	0.000	1000	0.0000000	0.0036821
UKE00105888	EDINBURGH: ROYAL BOTANIC GARDE	PRCP	0.671	1000	0.6418789	0.7001211
UKE00105930	BENMORE: YOUNGER BOTANIC GARDE	PRCP	0.000	1000	0.0000000	0.0036821

We want to test the following hypothesis:

H_0 : The precipitation distribution is the same in winter as in summer

H_1 : The winter and summer distributions have different expected values

We use the absolute value of test statistic as follows, separately for each station:

$$T = |\text{winter average} - \text{summer average}|$$

First, we compute the statistic T_0 on the original data. Since we don't have a model for the data distributions in winter or summer, we perform a randomized permutation test. We add a Summer column to the data that is TRUE for data in April through September. We can then construct random permutations of this information within each station and compute the corresponding test statistics. Under the null hypothesis, the winter and summer observations are exchangeable, but not under the alternative hypothesis, so under the null hypothesis, we would expect the original test statistic to be distributionally indistinguishable from the randomized test statistics, but under the alternative hypothesis, we would expect the original test statistic to be larger, on average. We count the number of times $T_j \geq T_0$. The model for the counts and p-values is as follows:

$$X = \sum_{j=1}^{N_T} \mathbb{I}(T_j \geq T_0) \sim \text{Bin}(N_T, p), \quad p = P(T_j \geq T_0 \mid H_0),$$

During the test, our focus is on small values of p , as they would prompt us to reject the null hypothesis. While we cannot access p directly, we can estimate it using $\hat{p} = x/N_T$, where x denotes the observed counts.

Given that $N_T p(1-p)$ is large, a 95% confidence interval for a p-value can be approximated by

$$\hat{p} \pm 2 \cdot \sqrt{\frac{\hat{p}(1-\hat{p})}{N_T}}.$$

Since most of our observed counts equal zero, we employ the alternative interval $[0, 1 - 0.025^{1/N_T}]$ in such cases. We dismiss the null hypothesis H_0 if the related p-value confidence interval falls entirely below a chosen significance threshold. However, it is crucial to acknowledge that the actual p-value could exceed this threshold. The `p_value_CI` function, which creates these intervals, also incorporates the Bayesian credible interval method, yielding very similar outcomes.

For test with $N_T = 1000$, the results reveal that except for two stations, the estimated p-values are zero, suggesting distinct seasonal patterns at the stations. In Edinburgh's case, the p-value is unexpectedly high, indicating that we cannot dismiss the null hypothesis of no seasonal differences. For Leuchars, the p-value estimate is positive, yet the confidence interval still implies a small value. This particular interval may

necessitate a larger N_T value to achieve true confidence. The Hints document's approximate bounds suggest that N_T may need to be as large as 10,000 for a 0.02 interval width for all potential p-values. Nevertheless, more nuanced reasoning, such as assuming p is smaller than 0.05, indicates that $N_T = 1000$ is likely adequate in this case. Although the resulting p-value interval for Edinburgh is broad, the lower endpoint is considerably large, so we do not require a more accurate estimate to determine that the true p-value is improbable to be below any reasonably chosen statistical significance threshold.

Spatial weather prediction

Estimation and prediction

As a fundamental model for the square root of average monthly precipitation, we define M_0 : Value_sqrt_avg $\sim$ Intercept + Longitude + Latitude + Elevation + DecYear. Covariates such as spatial coordinates, elevation, and appropriate cos and sin functions are used to capture seasonal variability. By incorporating covariates $\cos(2\pi kt)$ and $\sin(2\pi kt)$ with frequency $k = 1, 2, \dots$, we can also model the seasonal variability, defining models M_1, M_2, M_3 , and M_4 . The predictor expression for model M_K adds the following term to the M_0 predictor expression:

$$\sum_{k=1}^K [\gamma_{c,k} \cos(2\pi kt) + \gamma_{s,k} \sin(2\pi kt)]$$

to the M_0 predictor expression, for model coefficients $\gamma_{c,k}$ and $\gamma_{s,k}$. The time variable t is defined to be DecYear so that the lowest frequency $k = 1$ corresponds to a cosine function with a period of one full year.

Estimate the model parameters for M0, M1, M2, M3

and M4.

```
# Define different model configurations
models <- list(
  M0 = list(order = 0, interaction = FALSE),
  M1 = list(order = 1, interaction = FALSE),
  M2 = list(order = 2, interaction = FALSE),
  M3 = list(order = 3, interaction = FALSE),
  M4 = list(order = 4, interaction = FALSE)
)

# Construct model formulas for each model configuration
formulas <- lapply(
  models,
  function(mod) {
    Model_formula(order = mod$order,
      extra = if (mod$interaction) {
        "I(cos1*(DecYear - mean(DecYear))) + I(sin1*(DecYear - mean(DecYear)))"
      } else {
        NULL
      })
  })

# Initialize lists to store joint model estimates and scores
joint_fit <- list()
cross_scores <- list()

# Rename ghcnd_mon to avoid confusion
climate_data <- incorporate_harmonics(ghcnd_mon, harmonic_order = 4)

# Loop through each model configuration
for (mod in names(models)) {
  form <- formulas[[mod]]

  # Estimate linear regression model using data (climate_data) and the model formula
  joint_fit[[mod]] <- lm(form, data = climate_data)

  # Compute plain and cross-validated predictions
  pred_cross <- CV_predictions(climate_data, form)

  # Compute scores for plain and cross-validated predictions
  cross_scores[[mod]] <- cbind(Model = mod,
    order = models[[mod]]$order,
    interaction = models[[mod]]$interaction,
    climate_data,
    pred_cross,
    calculate_scores(climate_data, pred_cross))
}

# Combine cross_scores data frames row-wise
combined_cross_scores <- do.call(rbind, cross_scores)

# Save models, formulas, joint_fit, scores_plain, and combined_cross_scores as RDS files
saveRDS(models, "data/models.rds")
saveRDS(formulas, "data/formulas.rds")
```

```
saveRDS(joint_fit, "data/joint_fit.rds")
saveRDS(combined_cross_scores, "data/combined_cross_scores.rds")

# Display the summary for each model
for (mod in names(joint_fit)) {
  cat("\nModel:", mod, "\n")
  print(summary(joint_fit[[mod]]))
}
```

```
##
## Model: M0
##
## Call:
## lm(formula = form, data = climate_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.94594 -0.32642 -0.02307  0.30481  1.95082
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.477e+00  1.394e+00   1.777   0.0757 .
## DecYear      2.411e-03  3.968e-04   6.076 1.31e-09 ***
## Latitude     -1.360e-01  2.018e-02  -6.737 1.79e-11 ***
## Longitude    -5.464e-01  1.073e-02 -50.930 < 2e-16 ***
## Elevation     4.022e-04  7.689e-05   5.231 1.75e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4902 on 5442 degrees of freedom
## Multiple R-squared:  0.3424, Adjusted R-squared:  0.3419
## F-statistic: 708.3 on 4 and 5442 DF,  p-value: < 2.2e-16
##
##
## Model: M1
##
## Call:
## lm(formula = form, data = climate_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.98638 -0.30492 -0.01175  0.29335  1.89615
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.490e+00  1.319e+00   1.888   0.0591 .
## DecYear      2.376e-03  3.754e-04   6.329 2.67e-10 ***
## Latitude     -1.350e-01  1.909e-02  -7.069 1.76e-12 ***
## Longitude    -5.467e-01  1.015e-02 -53.862 < 2e-16 ***
## Elevation     4.040e-04  7.274e-05   5.554 2.92e-08 ***
## cos1         1.828e-01  8.902e-03  20.536 < 2e-16 ***
## sin1        -1.320e-01  8.870e-03 -14.885 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4637 on 5440 degrees of freedom
## Multiple R-squared:  0.4117, Adjusted R-squared:  0.411
## F-statistic: 634.5 on 6 and 5440 DF,  p-value: < 2.2e-16
##
##
## Model: M2
##
## Call:
## lm(formula = form, data = climate_data)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.98846 -0.30883 -0.01187  0.29641  1.89402
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.477e+00  1.318e+00   1.879 0.060239 .
## DecYear      2.382e-03  3.750e-04   6.351 2.31e-10 ***
## Latitude     -1.349e-01  1.907e-02  -7.075 1.69e-12 ***
## Longitude    -5.467e-01  1.014e-02 -53.911 < 2e-16 ***
## Elevation     4.042e-04  7.267e-05   5.562 2.80e-08 ***
## cos1         1.829e-01  8.894e-03  20.567 < 2e-16 ***
## sin1        -1.321e-01  8.862e-03 -14.903 < 2e-16 ***
## cos2         3.088e-02  8.889e-03   3.474 0.000516 ***
## sin2         7.686e-04  8.866e-03   0.087 0.930922
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4633 on 5438 degrees of freedom
## Multiple R-squared:  0.413, Adjusted R-squared:  0.4121
## F-statistic: 478.3 on 8 and 5438 DF,  p-value: < 2.2e-16
##
##
## Model: M3
##
## Call:
## lm(formula = form, data = climate_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.01405 -0.30872 -0.01268  0.29559  1.86840
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.470e+00  1.317e+00   1.876 0.060694 .
## DecYear      2.385e-03  3.747e-04   6.364 2.12e-10 ***
## Latitude     -1.349e-01  1.906e-02  -7.080 1.62e-12 ***
## Longitude    -5.467e-01  1.013e-02 -53.958 < 2e-16 ***
## Elevation     4.044e-04  7.261e-05   5.570 2.67e-08 ***
## cos1         1.828e-01  8.887e-03  20.568 < 2e-16 ***
## sin1        -1.321e-01  8.855e-03 -14.915 < 2e-16 ***
## cos2         3.072e-02  8.882e-03   3.459 0.000546 ***
## sin2         5.964e-04  8.859e-03   0.067 0.946329
## cos3        -7.072e-03  8.868e-03  -0.797 0.425214
## sin3         2.854e-02  8.872e-03   3.217 0.001305 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4629 on 5436 degrees of freedom
## Multiple R-squared:  0.4142, Adjusted R-squared:  0.4131
## F-statistic: 384.3 on 10 and 5436 DF,  p-value: < 2.2e-16
##
##
## Model: M4
##
```

```
## Call:
## lm(formula = form, data = climate_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.00162 -0.30627 -0.01235  0.29501  1.88086
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.471e+00  1.317e+00   1.877  0.06056 .
## DecYear      2.385e-03  3.747e-04   6.366 2.09e-10 ***
## Latitude    -1.350e-01  1.906e-02  -7.083 1.59e-12 ***
## Longitude   -5.467e-01  1.013e-02 -53.968 < 2e-16 ***
## Elevation    4.042e-04  7.259e-05   5.568 2.70e-08 ***
## cos1         1.829e-01  8.886e-03  20.578 < 2e-16 ***
## sin1        -1.320e-01  8.853e-03 -14.916 < 2e-16 ***
## cos2         3.070e-02  8.880e-03   3.457 0.00055 ***
## sin2         6.781e-04  8.857e-03   0.077 0.93897
## cos3        -7.045e-03  8.867e-03  -0.795 0.42691
## sin3         2.849e-02  8.870e-03   3.212 0.00133 **
## cos4         1.356e-02  8.900e-03   1.524 0.12763
## sin4         1.223e-02  8.838e-03   1.384 0.16656
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4628 on 5434 degrees of freedom
## Multiple R-squared:  0.4146, Adjusted R-squared:  0.4134
## F-statistic: 320.8 on 12 and 5434 DF,  p-value: < 2.2e-16
```

Present and discuss the different models and the results of the model estimation.

To more effectively display the differences among models, we compute the pairwise score differences with respect to model M1, which employs a single set of cos/sin functions as its seasonal model. Additionally, we reorganize the data into a longer format, introducing a column named Score to indicate which score type (“SE” or “DS”) is stored in a new score value column, S. By arranging the models in descending order based on improving Dawid-Sebastiani cross-validation scores, it becomes evident that the models are primarily ordered by the number of parameters.

We also compute the mean scores (S) for each station, as well as the scores averaged for each month within each station. To further analyze the model performance, we calculate these averages with the reference model scores removed (dS) and determine the standard errors for both the average scores (S_se) and the average score differences (dS_se). This approach allows us to better understand the performance of each model compared to the reference model M1 and assess their relative improvement across different stations and months.

Model	S_DS	S_SE	dS_DS	dS_SE
M0	-0.1814883	0.2842539	0.0820230	0.0213068
M1	-0.2635113	0.2629471	0.0000000	0.0000000
M2	-0.2642104	0.2625877	-0.0006991	-0.0003595

Model	S_DS	S_SE	dS_DS	dS_SE
M3	-0.2656085	0.2622205	-0.0020972	-0.0007266
M4	-0.2659012	0.2621351	-0.0023899	-0.0008120

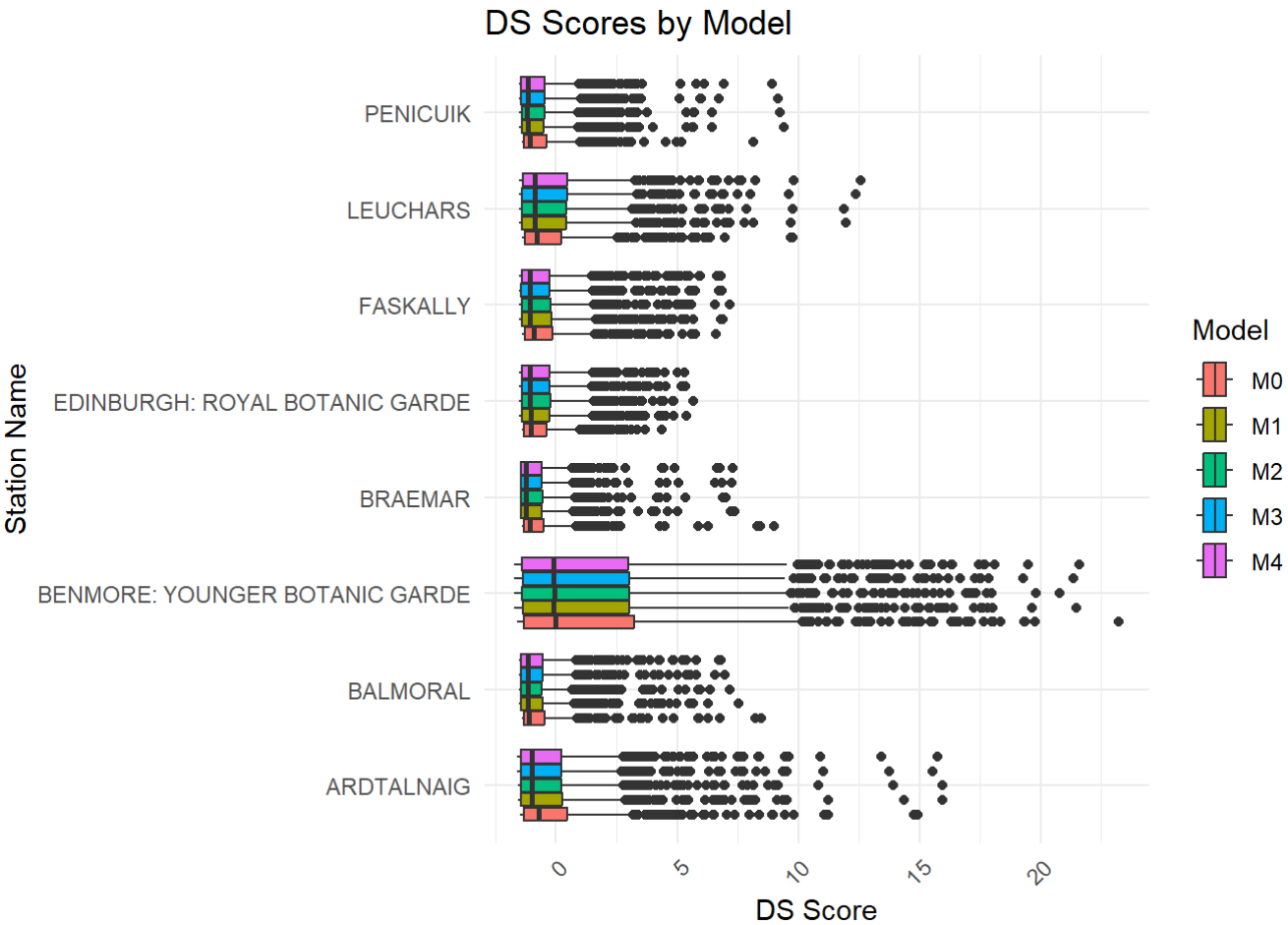
The most significant score improvement occurs between the non-seasonal model, M_0 , and the simplest seasonal model, M_1 . While the seasonal models all possess quite similar average scores, it is noteworthy that the M_1^I model already has a lower score than the M_3 model, illustrating that even a basic temporally changing seasonal model can enhance the model. However, given the small score differences between the seasonal models, the practical implications of this observation are uncertain.

Assessment: Station and season differences

Is the model equally good at predicting the different stations?

To evaluate the prediction accuracy across different stations, we create a plot illustrating the average cross-validation scores for each station. Additionally, we include intervals based on $\pm$ two times the standard error for the average score difference relative to the reference model. This allows us to determine if the model

differences are statistically smaller or larger than the differences between stations:



The two boxplots created using ggplot2 display the performance of different linear regression models (M0, M1, M2, M3, and M4) for a set of weather stations, based on two different scores: DS (Dawid-Sebastiani cross-validation scores) and SE (Squared Error).

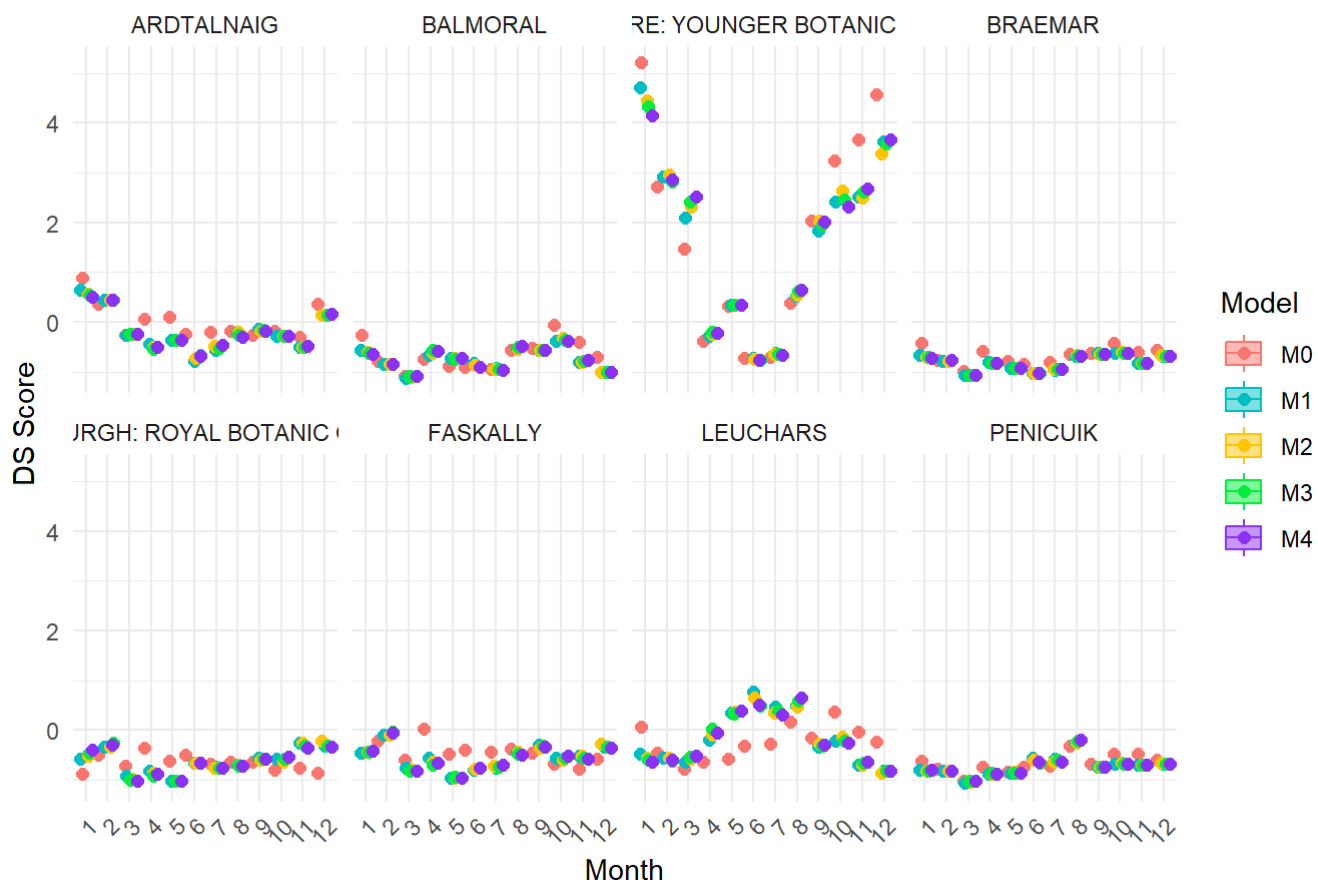
The first boxplot (DS_box_plot) illustrates the DS scores for each model across all weather stations. In this plot, you can observe that the non-seasonal M0 model consistently underperforms compared to the other models (M1, M2, M3, and M4). This suggests that adding seasonality and other factors in the models improves their ability to explain the variations in the data.

The second boxplot (SE_box_plot) shows the SE scores for each model across all weather stations. Similar to the DS boxplot, the non-seasonal M0 model appears to be worse than the other models, further confirming the importance of incorporating seasonality and other factors in the models.

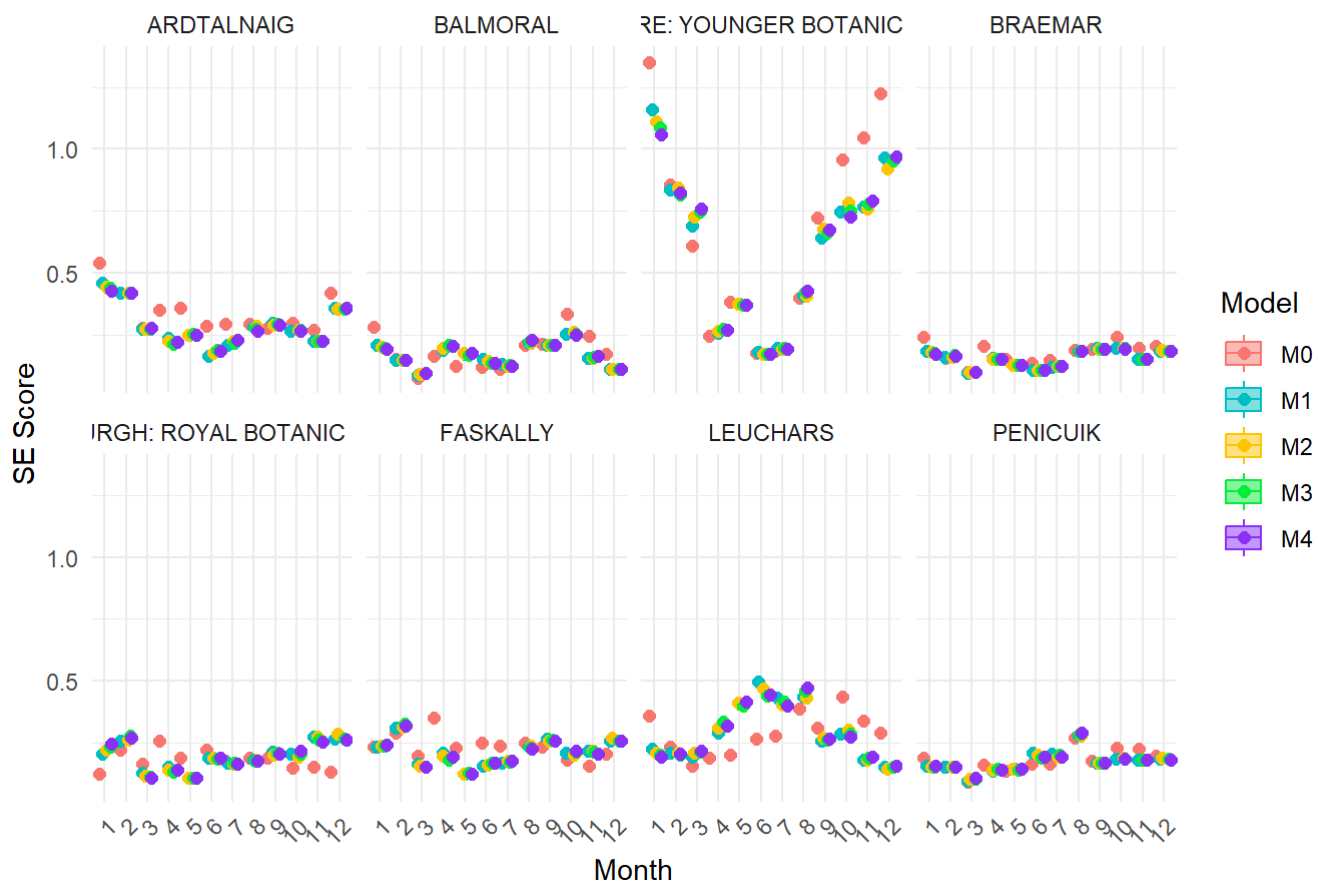
However, it is worth noting that the differences between stations are more significant than the average differences between models. This could indicate that the linear regression models are not capturing the nonlinear rainfall differences across space effectively. To better account for the spatial behavior in the data, more advanced spatial models would be required.

Is the prediction accuracy the same across the whole year?

Dawid-Sebastiani scores:



Squared Error scores:



The two sets of boxplots display the performance of different linear regression models (M0, M1, M2, M3, and M4) for a set of weather stations, based on two different scores, DS (Decomposition Score) and SE (Squared Error), and broken down by month. Custom colors are used to distinguish between the models.

The first set of boxplots (DS_station_box_plot) illustrates the DS scores for each model across all weather stations on a monthly basis. It's evident that the Benmore Botanic Garden station is particularly challenging to model with the basic models we're considering here. Benmore is located in the southwestern parts of the highlands, near the west coast, while the other stations are further inland and/or to the east.

The second set of boxplots (SE_station_box_plot) shows the SE scores for each model across all weather stations on a monthly basis. Similar to the DS boxplots, the performance of the models for the Benmore Botanic Garden station is noticeably worse compared to the other stations.

To improve the model's performance for a wider range of locations, including the Benmore Botanic Garden station, a more flexible seasonal model should be considered. This model would need to allow for non-linear variations in amplitude across space, thereby better capturing the unique spatial characteristics of each weather station.

Code appendix

Function definitions

```
# Chenrui Xu

# Place your function definitions that may be needed in the report.Rmd, including function docu-
# mentation.
# You can also include any needed library() calls here
suppressPackageStartupMessages(library(StatCompLab))
suppressPackageStartupMessages(library(tidyverse))

#' Seasonal Permutation Test Statistics
#'
#' Calculates permutation test statistics to compare equal versus non-equal distributions
#' during summer and winter months. This function handles both original and randomized test
#' statistics to prevent the need for double group and ungroup operations.
#'
#' @param data A data.frame containing at least the variables ID, Name,
#' Element, Value, and Summer (a logical variable indicating data from summer months).
#' Note: In the project, only precipitation data is used, but the function is designed to
#' allow the same analysis for TMAX and TMIN.
#' @param random A logical variable; if TRUE, the Summer information is
#' randomized groupwise within (ID, Name, Element)
#'
#' @return A data.frame with ID, Name, Element,
#' Test (test statistics for equal/non-equal mean)

seasonal_permutation_test_statistics <- function(data, random = FALSE) {
  data <- data %>%
    group_by(ID, Name, Element) %>%
    mutate(
      Summer =
        if (random) {
          sample(Summer, length(Summer), replace = FALSE)
        } else {
          Summer
        }
    ) %>%
    summarise(
      Test = abs(mean(Value[!Summer]) - mean(Value[Summer])),
      .groups = "drop"
    )
}

#' P-value Confidence Interval
#'
#' Designed to be called rowwise, so it provides only a single p_value estimate.
#'
#' @param p_value A p-value estimate derived from a binomial count proportion.
#' @param N The number of trials used to compute the estimate.
#' @param alpha The nominal interval error probability. The nominal coverage is
#' 1 - alpha.
#'
#' @return A data.frame with columns lwr (lower bound) and upr (upper bound).
p_value_confidence_interval <- function(p_value, N, alpha = 0.05) {
```

```

    if (p_value == 0) {
      data.frame(lwr = 0,
                 upr = 1 - (alpha / 2)^(1 / N))
    } else {
      se <- sqrt(p_value * (1 - p_value) / N)
      data.frame(lwr = p_value - qnorm(1 - alpha / 2) * se,
                 upr = p_value + qnorm(alpha / 2) * se)
    }
  }
}

#' Incorporate Harmonics into a Dataset
#'
#' @param dataset A data.frame with at least a DecYear column.
#' @param harmonic_order A non-negative integer specifying the number of harmonic
#' orders to be included in the seasonal model.
#'
#' @return A modified version of dataset with Cos1, Sin1, etc. added as
#' seasonal cycle base functions from orders 1 through harmonic_order, if harmonic_order is
#' greater than zero.
incorporate_harmonics <- function(data, harmonic_order = 1) {
  for (k in seq_len(harmonic_order)) {
    data[[paste0("cos", k)]] <- cos(2 * pi * data$DecYear * k)
    data[[paste0("sin", k)]] <- sin(2 * pi * data$DecYear * k)
  }
  data
}

#' Create Model Formula
#'
#' @param order A non-negative integer specifying the harmonic
#' orders to be integrated into the seasonal model. Utilize add_harmonics to
#' append the required covariates to the dataset.
#' @param extra A string containing supplementary model components.
#'
#' @return A formula designed for modeling Value as a dependent variable of DecYear,
#' Latitude, Longitude, Elevation, and optional seasonal harmonics, if applicable.
Model_formula <- function(order = 1, extra = NULL) {
  formula <- "Value ~ 1 + DecYear + Latitude + Longitude + Elevation"
  if (order > 0) {
    formula <- paste(c(formula,
                       paste0("cos", seq_len(order), " + sin", seq_len(order))),
                    collapse = " + ")
  }
  as.formula(paste0(c(formula, extra), collapse = " + "))
}

#' Cross-validated predictions
#'
#' @param data A data.frame containing observation data
#' @param formula A formula to be used with `lm()`
#'
#' @return A data.frame with columns `mean` and `sd` representing

```

```

#' the prediction mean and prediction standard deviation for each observation,
#' calculated using leave-station-out estimation and prediction for each station.

CV_predictions <- function(data, form) {
  result <- data.frame(
    mean = numeric(nrow(data)),
    sd = numeric(nrow(data))
  )
  for (id in unique(data$ID)) {
    fit <- lm(form, data %>% filter(ID != id))
    pred <- predict(fit, newdata = data %>% filter(ID == id), se.fit = TRUE)
    result$mean[data$ID == id] <- pred$fit
    result$sd[data$ID == id] <- sqrt(pred$se.fit^2 + pred$residual.scale^2)
  }
  result
}

#' Calculate Scores
#'
#' @param data A data.frame with a column named Value, at the very least.
#' @param pred A data.frame that contains columns mean and sd, at the very least.
#'
#' @return A data.frame containing columns SE and DS, which represent the squared error and
#' Dawid-Sebastiani scores, respectively. These scores are calculated by predicting pred$Value
#' using the prediction means (pred$mean) and standard deviations (pred$sd).
calculate_scores <- function(data, pred) {
  data.frame(
    SE = proper_score(
      type = "se", obs = data$Value,
      mean = pred$mean
    ),
    DS = proper_score(
      type = "ds", obs = data$Value,
      mean = pred$mean, sd = pred$sd
    )
  )
}

```